

CloudWatch Logs, metric filters and alarms

Turning a stream of log events into somebody's phone ringing – and centralising it across hundreds of accounts.

A log group is **a place logs go**. Nothing about creating one causes anyone to be told anything.

Anatomy of a log group

- ◆ **Retention defaults to Never expire.** A cost problem and a compliance problem at the same time.
- ◆ Streams sit inside a group. Retention, encryption and class are set on the **group**, never the stream.
- ◆ **Encrypted at rest by default** with an AWS owned key. A customer managed key is optional.
- ◆ From May 2026, Logs Insights can query every group sharing a **tag**.
- ◆ Metric filters are **not retroactive**. They see only events ingested after they exist.

ON THE EXAM

Never expire is the default. Most CloudWatch Logs cost questions start there.

Two log classes, and the difference is not just price

CAPABILITY	STANDARD	INFREQUENT ACCESS
Ingestion price	Baseline	About 50% lower
Logs Insights queries	Yes	Yes, plus OpenSearch PPL and SQL
Data protection masking	Yes	Yes, added March 2026
Metric filters and alarms	Yes	NO
Live Tail	Yes	NO

ON THE EXAM

Infrequent Access is a fine choice for forensic archives. Put a log group you alarm on in it and the alarm silently never fires.

Encrypting a log group with your own key

- ◆ Associate a **symmetric** customer managed key. Asymmetric keys are rejected.
- ◆ The key policy must let **logs.<region>.amazonaws.com** Encrypt, Decrypt, ReEncrypt, GenerateDataKey and Describe.
- ◆ Scope it with **kms:EncryptionContext:aws:logs:arn** so the key serves only the log groups you intend.
- ◆ Disable or schedule deletion and **ingestion stops and existing data becomes unreadable**.
- ◆ The service principal is **regional**. A policy from one Region will not serve another.

ON THE EXAM

The encryption context condition is how you stop a service key being used more broadly.

The chain, link by link

01

Pattern

Write a filter pattern. {
\$.eventName =
"DisableKey" }

02

Filter

The metric filter tests every event ingested into that group.

03

Metric

A match publishes a value to a custom CloudWatch metric.

04

Alarm

An alarm on that metric, threshold 1, period 1 minute.

05

Notify

The alarm action publishes to SNS, or invokes a Lambda.

ON THE EXAM

Metric filters only see events ingested after they exist. Creating one during an incident tells you nothing about the incident.

The alarms a security baseline expects

- ◆ **Root user activity**, and console sign-in without MFA. The two that appear in almost every benchmark.
- ◆ **Unauthorised API calls** – errorCode of AccessDenied or UnauthorizedOperation, at volume.
- ◆ **Changes to the controls themselves** – CloudTrail configuration, AWS Config, IAM policies, S3 bucket policies.
- ◆ **DisableKey and ScheduleKeyDeletion** on a customer managed key. The quiet start of a ransomware story.
- ◆ **Network changes** – security groups, network ACLs, route tables, internet gateways.

ON THE EXAM

Notice the theme: three of the five watch the security controls, not the workload. An attacker turns off the lights first.

Metric filter and alarm, or EventBridge rule?

METRIC FILTER AND ALARM WHEN

You want a threshold

- Rate matters – 50 denials, not one
- You want a dashboard and a graph
- The source is log text, not an API event
- A benchmark control names this pattern

EVENTBRIDGE RULE WHEN

You want a response

- One occurrence must trigger action
- Lambda or Step Functions must run
- You need the full event as input
- Latency matters – no metric round trip

THE TIEBREAKER

Counting is an alarm. Reacting is EventBridge. If the requirement says 'automatically remediate', it is EventBridge.

Subscription filters: moving logs somewhere else

- ◆ Near real-time delivery to **Kinesis Data Streams, Amazon Data Firehose, Lambda** or **OpenSearch Service**.
- ◆ Cross-account works through a **destination** in the receiving account, which carries its own **destination access policy**.
- ◆ Write that policy with **aws:PrincipalOrgID** instead of a list of account IDs. New accounts then work with no policy change.
- ◆ Firehose to S3 is the standard route into a SIEM or into Amazon Security Lake.
- ◆ This is a **copy**. You now pay to store the logs twice and must secure both copies.

ON THE EXAM

Subscription filter is the answer whenever logs must leave CloudWatch Logs and arrive somewhere else continuously.

Ship the logs, or query them where they are?

SUBSCRIPTION FILTERS WHEN

The logs must move

- A SIEM or third party is the consumer
- You need continuous, near real-time delivery
- Long retention outside CloudWatch Logs
- Transformation or enrichment in flight

CROSS-ACCOUNT OBSERVABILITY WHEN

The analysts must reach

- Sink in the monitoring account, links from sources
- A sink policy grants source accounts access
- Search and query in place, no copy, no pipeline
- Filter which log groups are shared

THE TIEBREAKER

Does data need to arrive somewhere, or do people need to see across accounts?
Delivery is a subscription filter; visibility is OAM.

Logs Insights when you are investigating

- ◆ **fields, filter, stats, sort, limit, parse.** Enough for most incident questions.
- ◆ **JOIN and sub-queries**, April 2026. Correlate security events across services and log groups.
- ◆ **lookup**, March 2026. Enriches raw IDs into meaningful names at query time.
- ◆ Query by **log group tag**, May 2026. Matching groups join automatically.
- ◆ It reads CloudWatch Logs only. Years of history means CloudTrail Lake, Athena or Security Lake.

ON THE EXAM

Logs Insights answers 'what happened this week', not 'what happened in 2023'.

A bank runs a dedicated log group for application authentication events. A metric filter matches failed logins and an alarm is configured at a threshold of 20 in five minutes, publishing to an SNS topic the on-call team subscribes to.

During a credential stuffing attack the log group records over 40,000 failed logins in an hour. The alarm never leaves INSUFFICIENT_DATA and nobody is paged. The filter pattern is correct and the SNS topic works when tested manually. The log group was moved to the Infrequent Access class last quarter to reduce cost.

Why did the alarm not fire?

- ☐ A The alarm threshold of 20 in five minutes was never reached
- ☐ B The metric filter was created before the log group was moved, so it needs recreating
- ☐ C The Infrequent Access log class does not support metric extraction
- ☐ D The log group's retention period expired the events before the alarm evaluated

◆ PAUSE AND DECIDE

Why did the alarm not fire?

- A

The threshold was never reached

40,000 in an hour is well past 20 in five minutes. And a breached threshold gives ALARM; an unbreached one gives OK. Neither is INSUFFICIENT_DATA.
- B

The filter needs recreating after the class change

Tempting because it is nearly true. Recreating changes nothing: the class does not support metric extraction at all, so a new filter would be equally silent.
- C

Infrequent Access does not support metric extraction

Correct. No metric extraction means no data points, which is exactly what INSUFFICIENT_DATA reports. The cost saving quietly removed the alerting.
- D

Retention expired the events first

Retention governs how long events are stored. Metric filters evaluate on ingestion, long before retention is relevant.

THE SIGNAL

INSUFFICIENT_DATA is the tell. Not breached, not clear – the metric has no data points, so nothing is publishing to it.

A retailer has 300 accounts in AWS Organizations and a commercial SIEM running outside AWS. Application and VPC flow logs land in CloudWatch Logs in each account. The SIEM must receive them continuously, within minutes, in a format its ingestion pipeline already understands.

New accounts are created roughly weekly. The security team has been editing an account ID list by hand every time and has missed several accounts.

What should the security team implement?

- A Create a CloudWatch cross-account observability sink in the security account and link all 300 accounts
- B Create a subscription filter in each account to a cross-account destination backed by Firehose, with a destination policy using `aws:PrincipalOrgID`
- C Schedule CloudWatch Logs export tasks to a central S3 bucket and have the SIEM poll the bucket
- D Configure the CloudWatch agent in each account to write directly to the SIEM endpoint

What should the security team implement?

- A

Cross-account observability sink and links
OAM lets analysts query across accounts. It does not deliver anything to an external SIEM. The logs never leave CloudWatch Logs.
- B

Subscription filters to a cross-account destination, policy on `aws:PrincipalOrgID`
Correct. Near real-time delivery, Firehose transforms and batches for the SIEM, and the org-ID condition means new accounts work without editing anything.
- C

Scheduled export tasks to S3
Export tasks are batch, manual and limited to one at a time per account. 'Continuously, within minutes' rules them out.
- D

CloudWatch agent writing directly to the SIEM
The agent's job is to get logs into CloudWatch Logs, not out of it. And VPC flow logs never touch the agent.

THE SIGNAL

'Editing an account ID list by hand' is not background. It is the question asking which condition key removes that work.

A different company, 40 accounts, has no external SIEM. During incidents its analysts need to search application log groups across every account from one console, and correlate them with metrics. Investigations happen perhaps twice a month.

The CISO has explicitly rejected a proposal to copy all logs into a central account, citing duplicated storage cost and a second copy of sensitive data to secure. Only log groups tagged Environment=prod should be visible centrally.

What should the company implement?

- ☐ A A CloudWatch cross-account observability sink in a monitoring account, with links from each source account filtered to the tagged log groups
- ☐ B Subscription filters in all 40 accounts delivering to a central Firehose and S3 bucket
- ☐ C An organization CloudTrail with a CloudTrail Lake event data store
- ☐ D IAM roles in each account that analysts assume one at a time to read log groups

◆ PAUSE AND DECIDE

What should the company implement?

- A** Cross-account observability sink with filtered links

Correct. Query in place with no copy, metrics and logs together in one console, and resource filtering restricts sharing to the tagged groups.

- C** Organization trail and CloudTrail Lake event data store

CloudTrail records API calls. The requirement is application log groups, which a trail never sees. Right service, wrong data.

- B** Subscription filters to a central Firehose and S3
This was the right answer in scenario 2 and is wrong here. The CISO rejected exactly this: it creates the duplicate copy and the duplicate cost.

- D** Analysts assume a role per account
It works and it is what they are escaping. Forty console switches during an incident, and no correlation across accounts.

THE SIGNAL

Same shape as scenario 2, one constraint changed. 'No copy' and 'no external consumer' flip the answer from delivery to visibility.

A healthcare provider encrypts its CloudWatch log groups with a customer managed KMS key. Following an audit finding about over-broad key policies, an engineer tightens the key policy so the CloudWatch Logs service principal can only use the key for log groups in the audited account.

Within an hour, log groups in that account stop receiving events. Existing log data can no longer be read in the console. CloudTrail shows the service principal receiving AccessDenied from KMS. The key is enabled and not scheduled for deletion.

What is the MOST likely cause?

- ☐ A The key was changed to asymmetric, which CloudWatch Logs does not support
- ☐ B The log groups must be re-associated with the key after any key policy change
- ☐ C The engineer removed kms:Decrypt, so writes succeed but reads fail
- ☐ D The encryption context condition references a Region or account that does not match the log groups

◆ PAUSE AND DECIDE

What is the MOST likely cause?

- A** The key was changed to asymmetric
You cannot convert a key's type after creation, and an asymmetric key could never have been associated in the first place. The scenario describes a policy change, not a new key.
- B** Log groups need re-associating after a policy change
There is no such requirement. The association is to the key ARN and survives policy edits. Plausible-sounding invented mechanics are a common distractor shape.
- C** kms:Decrypt was removed, so writes succeed but reads fail
Half right, and the half that is wrong matters. Ingestion also stopped, so encrypt is failing too. A missing Decrypt alone would not stop writes.
- D** The encryption context condition does not match the log groups
Correct. The condition pins the key to log group ARNs, which embed account and Region. Get either wrong and every operation is denied, reads and writes alike.

THE SIGNAL

Reads AND writes both failing points at a condition on the whole grant, not a missing action. Narrow the cause by what still works.

Five things to carry forward

- ◆ **Infrequent Access has no metric extraction.** The alarm never fires and never errors.
- ◆ **Metric filters are not retroactive.** INSUFFICIENT_DATA means no data points, not a missed threshold.
- ◆ **Counting is an alarm. Reacting is EventBridge.** 'Automatically remediate' is never an alarm.
- ◆ **Delivery is a subscription filter. Visibility is cross-account observability.**
- ◆ **aws:PrincipalOrgID beats a list of account IDs** in any policy across an organisation.

ON THE EXAM

Next: GuardDuty – and why enabling it is not the same as being covered.